LAPORAN
WORKSHOP Mobile Web Server
(TIF310808)
Acara 24
SEMESTER



Nama Tugas
Integrasi API dengan frontend

Nama:
Adam Ridho Saifulloh

NIM:
E41250216

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)
TEKNIK INFORMATIKA KAB. NGANJUK
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER
TAHUN 2026

A. PENDAHULUAN

Fetch API menyediakan antarmuka JavaScript modern untuk melakukan permintaan HTTP dan memproses response, menggantikan XMLHttpRequest yang berbasis callback. Pemanggilan fetch() mengembalikan sebuah Promise yang selesai begitu server merespons dengan header, sedangkan pembacaan body response (misalnya melalui method .json()) dilakukan pada tahap terpisah. Sintaks async/await memungkinkan kode asynchronous ditulis berurutan sehingga lebih mudah dibaca dibandingkan rangkaian .then(), dan kegagalan permintaan (misalnya jaringan terputus) ditangani dengan blok try-catch.

Cross-Origin Resource Sharing (CORS) adalah mekanisme keamanan berbasis header HTTP yang mengatur apakah halaman web pada suatu origin (domain, skema, atau port) boleh mengakses resource dari origin lain. Browser menolak response dari server lain kecuali server secara eksplisit mengirim header seperti Access-Control-Allow-Origin.

Praktik Acara 24 bertujuan menghubungkan REST API yang telah dibangun dengan antarmuka web (frontend): header CORS ditambahkan sekali pada Front Controller, kemudian sebuah halaman HTML menampilkan data mahasiswa dari endpoint GET /mahasiswa menggunakan fetch().

B. PROSEDUR KERJA

Langkah 1 — Mengaktifkan Header CORS pada Front Controller

Header CORS ditambahkan pada awal public/index.php sebelum Router dijalankan, sehingga berlaku otomatis untuk seluruh resource (mahasiswa, buku, login, register). Selain tiga header pada modul, ditambahkan penanganan preflight request (method OPTIONS) agar browser dapat mengirim PUT, DELETE, maupun header Authorization dari origin lain.

```php
<?php
// Header CORS (Acara 24): berlaku untuk seluruh resource yang melewati
Router.
header('Access-Control-Allow-Origin: *');
header('Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS');
header('Access-Control-Allow-Headers: Content-Type, Authorization');

// Preflight request dari browser (cross-origin dengan header Authorization /
method PUT, DELETE).
if (($_SERVER['REQUEST_METHOD'] ?? '') === 'OPTIONS') {
    http_response_code(204);
    exit;
}

require_once __DIR__ . '/../app/core/autoload.php';

(new Router())->dispatch();
```

Langkah 2 — Membuat Folder View

Folder views dibuat di dalam public dengan dua berkas, yaitu index.html dan app.js. Karena berada di dalam public, berkas ini disajikan langsung oleh Apache (aturan .htaccess hanya mengarahkan ke Router bila berkas tidak ada) dan tidak melewati Router.

```
public/
├── index.php
├── .htaccess
└── views/
    ├── index.html
    └── app.js
```

Langkah 3 — Menulis Kode Fetch untuk Menampilkan Data

Berkas app.js mengambil data dari GET /mahasiswa dengan async/await dan menampilkannya pada tabel. Alamat API diturunkan dari lokasi halaman sehingga tetap benar apa pun nama folder proyek.

```
async function muatDataMahasiswa() {
    // Tugas Mandiri 1: indikator sebelum data berhasil diambil.
    tampilkanPesan('Memuat data...');

    try {
        const response = await fetch(`${BASE_URL}/mahasiswa`);
        const hasil = await response.json();


        if (!response.ok || !hasil.success) {
            throw new Error(hasil.message || `HTTP ${response.status}`);
        }

        // Tugas Mandiri 2: kondisi data kosong.
        if (!Array.isArray(hasil.data) || hasil.data.length === 0) {
            tampilkanPesan('Belum ada data');
            return;
        }

        tbody.innerHTML = '';
        hasil.data.forEach((mhs) => {
            const tr = document.createElement('tr');
            [mhs.nim, mhs.nama, mhs.prodi].forEach((nilai) => {
                const td = document.createElement('td');
                td.textContent = nilai; // textContent mencegah XSS dari data
API
                tr.appendChild(td);
            });
            tbody.appendChild(tr);
        });
    } catch (error) {
        console.error('Gagal memuat data:', error);
```

```
        tampilkanPesan('Gagal memuat data. Periksa koneksi atau server API.',
true);
    }
}

document.addEventListener('DOMContentLoaded', muatDataMahasiswa);
```

Baris tabel dibentuk dengan createElement dan textContent, bukan innerHTML, sehingga data dari API yang mengandung tag HTML tidak dieksekusi sebagai skrip (mencegah XSS).

Langkah 4 — Membuat View index.html

Halaman memuat tabel dengan kolom NIM, Nama, dan Program Studi serta memanggil app.js.

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Data Mahasiswa</title>
  <style>
    body { font-family: Arial, sans-serif; max-width: 800px; margin: 2rem
auto; padding: 0 1rem; color: #222; }
    h1 { font-size: 1.5rem; }
    table { width: 100%; border-collapse: collapse; margin-top: 1rem; }
    th, td { border: 1px solid #ccc; padding: 8px 12px; text-align: left; }
    th { background: #f0f0f0; }
    .status td { text-align: center; color: #555; font-style: italic; }
    .status.error td { color: #b00020; font-style: normal; }
  </style>
</head>
<body>
  <h1>Data Mahasiswa</h1>
  <table id="tabel-mahasiswa">
    <thead>
      <tr><th>NIM</th><th>Nama</th><th>Program Studi</th></tr>
    </thead>
    <tbody></tbody>
  </table>
  <script src="app.js"></script>
</body>
</html>
```

Langkah 5 — Pengujian Tampilan View

- Halaman dibuka melalui browser pada alamat http://localhost/<path-proyek>/sistem-informasi-akademik/public/views/index.html (berkas statis, tidak melewati Router).
- Data pada tabel HTML dicocokkan dengan hasil MahasiswaController::index() pada Postman.

- Developer Tools (tab Network dan Console) diperiksa untuk memastikan tidak ada error CORS.

D. TUGAS PRAKTIK MANDIRI

1. Indikator "Memuat data..."

Fungsi tampilkanPesan() menampilkan satu baris pada tabel. Fungsi ini dipanggil dengan teks "Memuat data..." di awal muatDataMahasiswa(), sebelum fetch dijalankan, sehingga pengguna melihat indikator sampai data berhasil diambil.

```
async function muatDataMahasiswa() {
    // Tugas Mandiri 1: indikator sebelum data berhasil diambil.
    tampilkanPesan('Memuat data...');

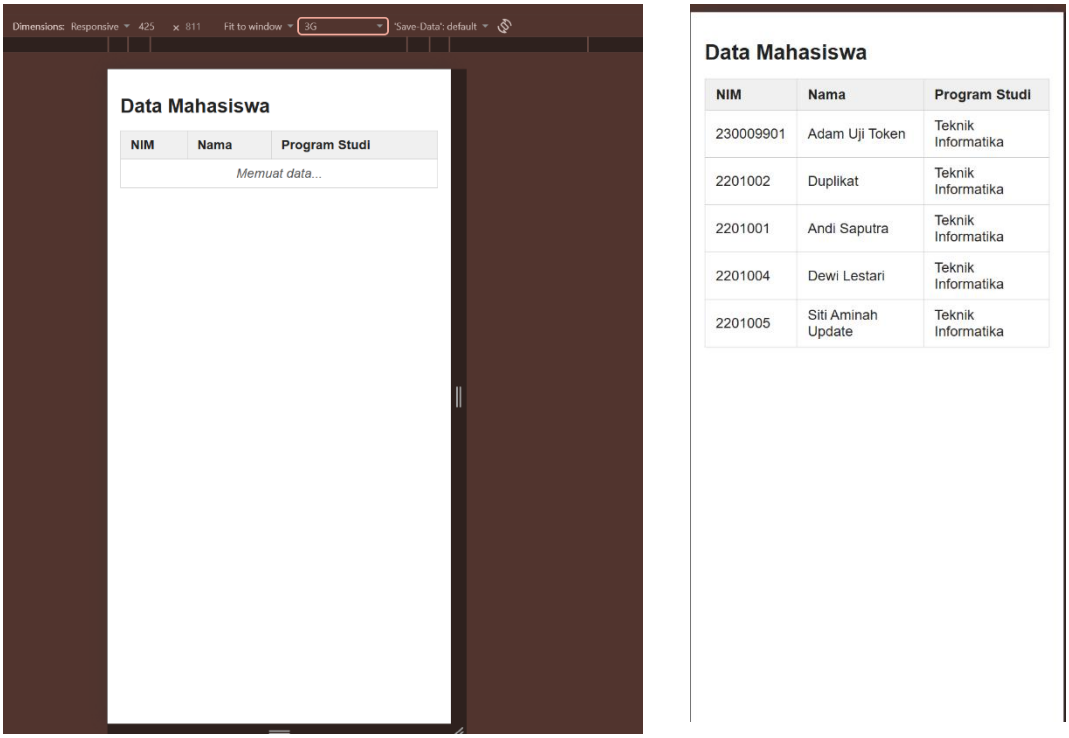
    try {
        const response = await fetch(`${BASE_URL}/mahasiswa`);
        const hasil = await response.json();


        if (!response.ok || !hasil.success) {
            throw new Error(hasil.message || `HTTP ${response.status}`);
        }

        // Tugas Mandiri 2: kondisi data kosong.
        if (!Array.isArray(hasil.data) || hasil.data.length === 0) {
            tampilkanPesan('Belum ada data');
            return;
        }

        tbody.innerHTML = '';
        hasil.data.forEach((mhs) => {
            const tr = document.createElement('tr');
            [mhs.nim, mhs.nama, mhs.prodi].forEach((nilai) => {
                const td = document.createElement('td');
                td.textContent = nilai; // textContent mencegah XSS dari data
API
                tr.appendChild(td);
            });
            tbody.appendChild(tr);
        });
    } catch (error) {
        console.error('Gagal memuat data:', error);
        tampilkanPesan('Gagal memuat data. Periksa koneksi atau server API.',
true);
    }
}

document.addEventListener('DOMContentLoaded', muatDataMahasiswa);
```



Gambar 5. Indikator "Memuat data..." (jaringan dibuat lambat melalui DevTools)

2. Penanganan data mahasiswa kosong

Setelah response diterima, bila hasil.data bukan array atau panjangnya nol, tabel menampilkan pesan "Belum ada data". Kondisi gagal (jaringan terputus atau response error) juga ditangani dengan pesan kesalahan pada tabel.

```
try {
        const response = await fetch(`${BASE_URL}/mahasiswa`);
        const hasil = await response.json();
        hasil.data = []; // SEMENTARA untuk uji data kosong
```



Gambar 6. Tampilan "Belum ada data" ketika tabel mahasiswa kosong

E. KESIMPULAN

Integrasi REST API dengan frontend berhasil dilakukan: header CORS dipasang satu kali pada Front Controller, dan halaman HTML mengambil data mahasiswa melalui fetch() dengan async/await serta menampilkannya pada tabel. Tugas mandiri melengkapi pengalaman pengguna dengan indikator "Memuat data..." dan pesan "Belum ada data", serta pesan kesalahan bila API tidak dapat dihubungi. Pemakaian textContent dan alamat API yang diturunkan dari lokasi halaman membuat view lebih aman dan portabel.